Microsoft Elevate

CAPSTONE PROJECT

AI Tool Discovery Agent

PRESENTED BY

STUDENT NAME: ADIT JAIN

COLLEGE NAME: SRM IST, DELHI NC CAMPUS

DEPARTMENT: B. TECH (CSE) - AIML

EMAIL ID: AJ9104@SRMIST.EDU.IN

OUTLINE:

- Problem Statement
- Proposed Solution
- Data Flow Pipeline
- Project Architecture
- Technology Stack
- Methodology
- Result
- Conclusion
- Future Scope
- References

PROBLEM STATEMENT:

- The AI ecosystem moves fast — new tools launch daily across hundreds of websites
- Manually tracking them is time-consuming, error-prone, and not scalable
- Search results are flooded with news articles, aggregators, social media, and duplicates
- Tool information is scattered across marketing pages with no consistent structure
- By the time a team discovers a new tool, competitors may already be using it

Question: How do you systematically discover every new AI tool within 7 days of launch, filter out noise, extract structured data, and deliver it to your team — automatically?

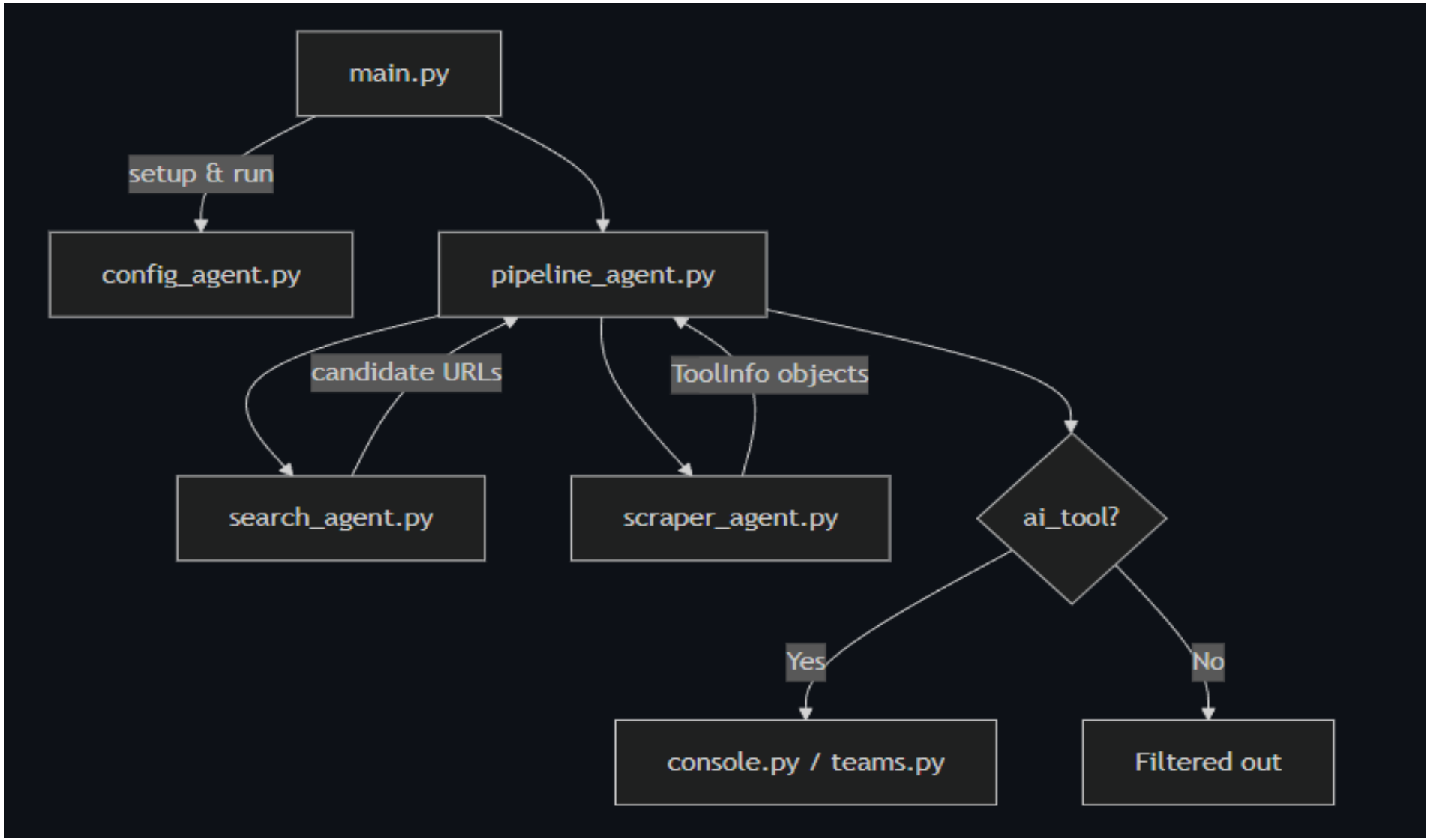
PROPOSED SOLUTION:

An end-to-end autonomous agent pipeline with zero manual intervention:

1. **SEARCH — Query two search engines with 6 query categories, strict 7-day filter**
2. **FILTER — Block 70+ aggregator/news/social domains, normalize and deduplicate URLs**
3. **SCRAPE — Async batch HTML fetching with robots.txt checks, retries, and blacklisting**
4. **ANALYZE — Single GPT-4o call per tool: extract all fields + classify as AI tool or not**
5. **DELIVER — Print structured results to console and send to Microsoft Teams via webhook**

Result: Only confirmed, newly launched AI tools — structured, classified, and delivered.

DATA FLOW PIPELINE:



Project Architecture:

main.py — Entry point, orchestration, Teams delivery
agents/config_agent.py — Load .env, validate API keys, setup logging
agents/search_agent.py — Dual search engine queries + filtering
agents/scraper_agent.py— Async HTML fetch + GPT-4o extraction
agents/pipeline_agent.py— Pipeline orchestration, batching, retries
prompts/ — system_prompt.txt + user_prompt.txt (LLM templates)
models/tool_info.py — ToolInfo dataclass (structured output)
utils/prompt_loader.py — Load prompt files + robots.txt checker
utils/error_handling.py— Custom exception hierarchy with context
utils/blacklist.py — Persistent domain blacklist (3-strike rule)
output/console.py — Console formatting and summary
output/teams.py — Teams webhook integration
config/constants.py — All tunable parameters in one place
data/blacklist.json — Auto-generated blacklisted domains

Technology Stack:

Language:	Python 3.9+
LLM:	Azure OpenAI GPT-4o
Search APIs:	Serper.dev + SerpAPI
Async HTTP:	aiohttp
LLM Client:	openai (AsyncAzureOpenAI)
Progress Bars:	tqdm
Config:	python-dotenv
Output:	Console + Microsoft Teams (webhook)
Testing:	pytest + pytest-asyncio
Compression:	brotli

10 dependencies in requirements.txt, all with minimum version pins.

METHODOLOGY:

Search: Two engines (Serper + SerpAPI) run in parallel across 6 query types.

- 70+ aggregator/news/social domains blocked. URLs normalized and deduplicated with strict
- 7-day recency filter.

Scraping: Async batch fetching (batch=4) with 3 retries and exponential backoff.

- robots.txt respected and cached. Persistent blacklist auto-skips domains after 3 failures.
- Custom exception hierarchy carries full context.

LLM: Single GPT-4o call per tool extracts 10 fields + classifies as AI tool or not.

- Prompts externalized. Semaphore-throttled (concurrency=2). Regex fallback for JSON parsing.
- HTML truncated to 15K chars.

Parallelism: Search, fetch, and LLM steps are all async and parallel. Fetch throttled

- by batch size, LLM by semaphore. All limits configurable in one file (constants.py).

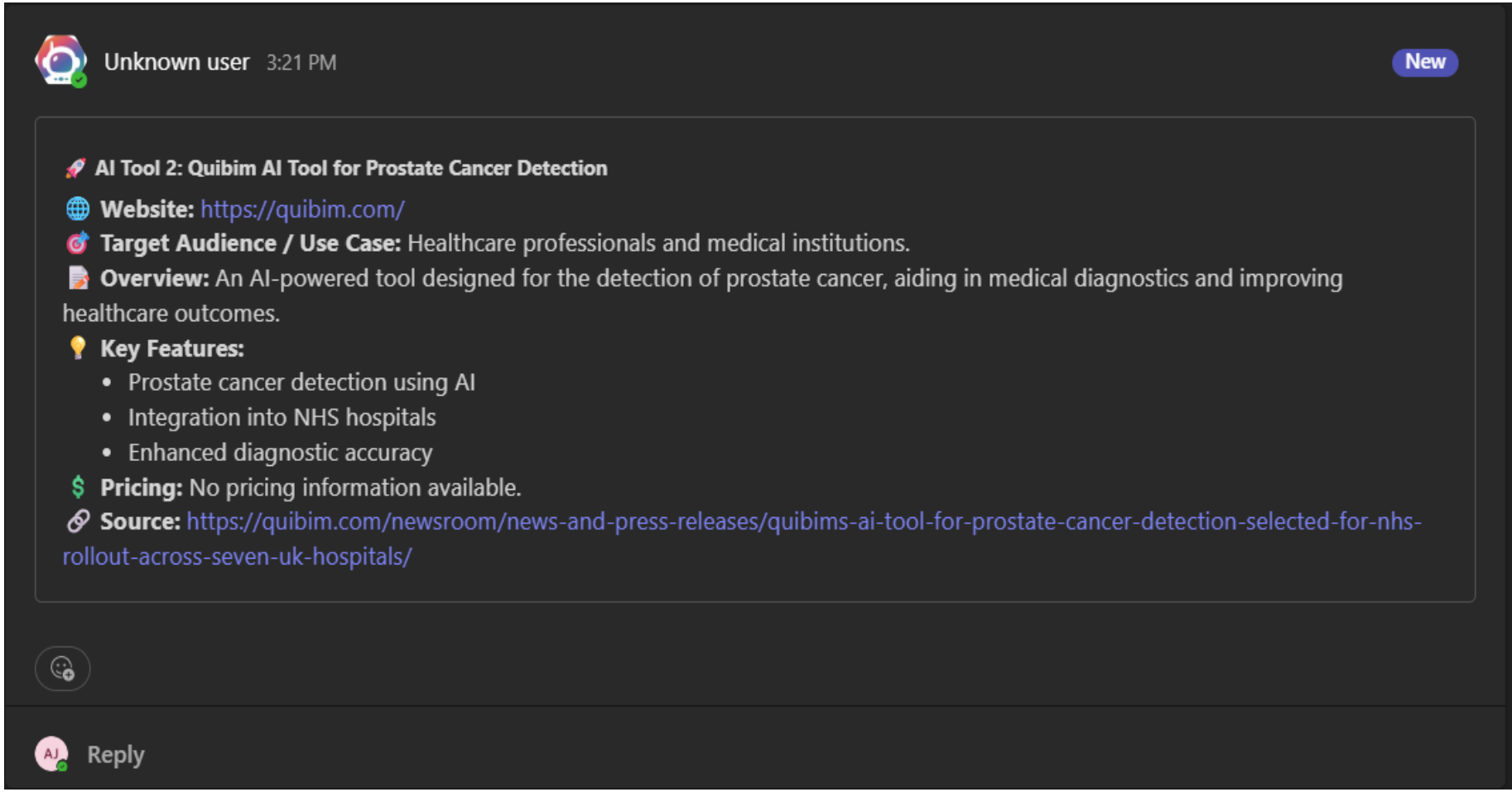
RESULT:

Console (output/console.py):

- Formatted per-tool display
- Summary: total URLs, successes, failures, skipped, blacklisted

Teams (output/teams.py):

- Rich markdown card per tool via webhook POST
- Includes: Title, Website, Summary, Features, Pricing, Audience, Tags, Date
- Only "ai_tool" classified results are sent
- Webhook URL from TEAMS_WEBHOOK_URL env variable



CONCLUSION:

- Fully autonomous — zero manual intervention
- Dual-engine search with 6 query categories
- 70+ aggregator domains blocked
- Async pipeline with batching, retries, backoff
- GPT-4o single-call extraction + classification
- Self-healing persistent blacklist
- Microsoft Teams delivery
- Modular, extensible architecture

FUTURE SCOPE:

- Additional search engines
- Multi-LLM support (Claude, Gemini fallback)
- Scheduled automation (cron/Azure Functions)
- Database persistence
- Web dashboard (Streamlit/React)
- Additional output channels (Slack, email, Notion)
- Smarter classification (fine-tuned smaller model)
- Multi-language & region support
- CLI & API interface
- Comparative analytics (weekly digest)

REFERENCES:

APIs & Services:

- Azure OpenAI Service — https://learn.microsoft.com/en-us/azure/ai-services/openai/
- Serper.dev (Google Search API) — https://serper.dev/ - SerpAPI (Search Engine Results API) — https://serpapi.com/
- Microsoft Teams Webhooks — https://learn.microsoft.com/en-us/microsoftteams/platform/webhooks-and-connectors/

Python Libraries:

- aiohttp (async HTTP client) — https://docs.aiohttp.org/
- openai (Python SDK) — https://github.com/openai/openai-python
- python-dotenv — https://github.com/theskumar/python-dotenv
- tqdm (progress bars) — https://github.com/tqdm/tqdm
- pytest / pytest-asyncio — https://docs.pytest.org/

Concepts & Techniques:

- Prompt Engineering for Structured Extraction — OpenAI Cookbook
- robots.txt Protocol — https://www.robotstxt.org/
- Exponential Backoff & Retry — Google Cloud Architecture Guide
- Async Programming in Python — https://docs.python.org/3/library/asyncio.html
- Dataclasses in Python — https://docs.python.org/3/library/dataclasses.html

GitHub Link: https://github.com/Adit-Jain-srm/AI-Tool-Discovery-Agent

Thank You